

LEVEL 19 — Linear Algebra with NumPy

Easy, line-by-line examples with outputs for Data Science and Machine Learning.

1. Matrix Basics

A matrix is a rectangular arrangement of numbers in rows and columns.

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

print(A)

Output:
[[1 2]
 [3 4]]
```

2. Matrix Creation

Create a matrix using `np.array()`.

```
import numpy as np

A = np.array([[10, 20, 30],
              [40, 50, 60]])

print(A)
print("Shape:", A.shape)

Output:
[[10 20 30]
 [40 50 60]]
Shape: (2, 3)
```

3. Matrix Addition

Add corresponding elements of two matrices.

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

print(A + B)

Output:
[[ 6  8]
 [10 12]]
```

4. Matrix Subtraction

Subtract corresponding elements.

```
A = np.array([[10, 20],
              [30, 40]])

B = np.array([[1, 2],
              [3, 4]])

print(A - B)
```

```
Output:
[[ 9 18]
 [27 36]]
```

5. Matrix Multiplication

Matrix multiplication uses rows of the first matrix and columns of the second matrix.

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

print(A @ B)
```

```
Output:
[[19 22]
 [43 50]]
```

6. Element-wise Multiplication

Multiply elements at the same positions.

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

print(A * B)
```

```
Output:
[[ 5 12]
 [21 32]]
```

7. Transpose

Transpose changes rows into columns.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print(A.T)
```

```
Output:
[[1 4]
 [2 5]
 [3 6]]
```

8. Identity Matrix

An identity matrix has 1s on the main diagonal and 0s elsewhere.

```
I = np.eye(3)

print(I)

Output:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

9. Diagonal Matrix

A diagonal matrix has values on the main diagonal and zeros elsewhere.

```
D = np.diag([10, 20, 30])
```

```
print(D)
```

```
Output:  
[[10  0  0]  
 [ 0 20  0]  
 [ 0  0 30]]
```

10. np.dot()

np.dot() performs a dot product. For 2-D matrices, it can perform matrix multiplication.

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
B = np.array([[5, 6],  
              [7, 8]])
```

```
print(np.dot(A, B))
```

```
Output:  
[[19 22]  
 [43 50]]
```

11. np.matmul()

np.matmul() performs matrix multiplication.

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
B = np.array([[5, 6],  
              [7, 8]])
```

```
print(np.matmul(A, B))
```

```
Output:  
[[19 22]  
 [43 50]]
```

12. np.linalg.det()

Finds the determinant of a square matrix.

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
print(np.linalg.det(A))
```

```
Output:  
-2.0
```

13. np.linalg.inv()

Finds the inverse of an invertible square matrix.

```
A = np.array([[1, 2],  
              [3, 4]])
```

```
print(np.linalg.inv(A))
```

```
Output:
[[-2.  1. ]
 [ 1.5 -0.5]]
```

14. np.linalg.solve()

Solves a system of linear equations. Here: $x + y = 5$ and $2x + 3y = 13$.

```
A = np.array([[1, 1],
              [2, 3]])

b = np.array([5, 13])

x = np.linalg.solve(A, b)

print(x)
```

```
Output:
[2. 3.]
```

15. np.linalg.eig()

Finds eigenvalues and eigenvectors.

```
A = np.array([[2, 0],
              [0, 3]])

values, vectors = np.linalg.eig(A)

print("Eigenvalues:", values)
print("Eigenvectors:")
print(vectors)
```

```
Output:
Eigenvalues: [2. 3.]
Eigenvectors:
[[1. 0.]
 [0. 1.]]
```

16. np.linalg.eigvals()

Finds only the eigenvalues.

```
A = np.array([[2, 0],
              [0, 3]])

print(np.linalg.eigvals(A))
```

```
Output:
[2. 3.]
```

17. np.linalg.matrix_rank()

Finds the rank of a matrix.

```
A = np.array([[1, 2],
              [2, 4]])

print(np.linalg.matrix_rank(A))
```

```
Output:
1
```

18. np.linalg.norm()

Calculates the norm (magnitude/length) of an array.

```
A = np.array([3, 4])  
  
print(np.linalg.norm(A))
```

Output:
5.0

19. np.linalg.svd()

SVD means Singular Value Decomposition. It decomposes a matrix into U, singular values, and Vt.

```
A = np.array([[1, 0],  
              [0, 2]])  
  
U, S, Vt = np.linalg.svd(A)  
  
print("U:")  
print(U)  
  
print("Singular values:")  
print(S)  
  
print("Vt:")  
print(Vt)
```

Output:
U:
[[0. 1.]
 [1. 0.]]
Singular values:
[2. 1.]
Vt:
[[0. 1.]
 [1. 0.]]

Note:
Signs of singular vectors may sometimes differ, but the decomposition remains valid.

Quick Revision — LEVEL 19

A + B — Matrix addition

A - B — Matrix subtraction

A * B — Element-wise multiplication

A @ B — Matrix multiplication

A.T — Transpose

np.eye() — Identity matrix

np.diag() — Diagonal matrix

np.dot() — Dot product / matrix multiplication for 2-D arrays

np.matmul() — Matrix multiplication

np.linalg.det() — Determinant

np.linalg.inv() — Inverse

np.linalg.solve() — Solve linear equations

np.linalg.eig() — Eigenvalues + eigenvectors

np.linalg.eigvals() — Eigenvalues only

np.linalg.matrix_rank() — Matrix rank

np.linalg.norm() — Vector/matrix norm

np.linalg.svd() — Singular Value Decomposition